OBJECT-ORIENTED PROGRAMMING PROJECT REPORT

ATM SIMULATOR

A secure, modular banking simulation built on classes, objects, inheritance and polymorphism.

6	3	7	5
menu operations	PIN attempts	OOP concepts	classes

SUBMITTED BY

Ranjan Kumar Nandi

Student ID **B125095** | CSE B(1) | 2nd year, 3rd semester

International Institute of Information Technology

CONTENTS

Table of Contents

1	Introduction	Page 3
2	Problem statement	Page 3
3	Abstract	Page 4
4	Objectives	Page 4
5	Features	Page 4
6	Requirements	Page 5
7	OOP concepts used	Page 6
8	Class design	Page 6
9	Module description	Page 7
10	Workflow	Page 8
11	Technology and requirements	Page 8
12	Operation algorithms	Page 9
13	Testing and validation	Page 10
14	Sample output	Page 10
15	Advantages	Page 11
16	Future scope	Page 11
17	Conclusion	Page 11

Acknowledgement

I sincerely thank my sir, [GUIDE NAME], for the guidance, the clear directions and the chance to build this project. I also thank my classmates and family for their constant support and encouragement while I prepared this report.

SECTIONS 1 AND 2

Introduction and Problem Statement

1 Introduction

An **Automated Teller Machine (ATM)** is a banking terminal that lets customers check balances, withdraw cash and deposit money at any hour without visiting a branch. Behind the simple screen sits a strict flow of identification, validation and record keeping.

This project builds a software model of that flow. The **ATM Simulator** uses Object-Oriented Programming to represent customers, accounts, transactions and the machine itself as cooperating objects, so the logic of a real ATM can be studied and tested safely on a computer.

2 Problem statement

Many students meet classes, objects, inheritance and encapsulation only as theory. They need a practical project where these ideas solve a real, familiar problem.

The ATM Simulator answers this need. It models a banking device that everyone understands, with no bank hardware and no real money at risk, while still demanding careful design, validation and error handling.

Scope of the project

Inside the scope

- Card and PIN login
- Six menu operations
- Input validation and error messages
- Card lock after 3 wrong PINs
- Transaction records and mini statement

Outside the scope

- Connection to a real bank network
- Physical ATM hardware
- Real money or real accounts
- Network security and encryption
- Multi-branch banking systems

Simple to use | **Safe to practise** | **Easy to extend**

SECTIONS 3 TO 5

Abstract, Objectives and Features

3 Abstract

An **Automated Teller Machine (ATM)** lets customers do basic banking without visiting a branch. The **ATM Simulator** is a software application built on Object-Oriented Programming that imitates a real ATM in a safe, virtual environment.

A user enters a card number, authenticates with a secret PIN, and then checks the balance, withdraws or deposits cash, changes the PIN or views a mini statement. Every action is validated, so wrong PINs, insufficient balance and bad inputs get clear messages.

Balance and PIN stay private inside the classes and are reachable only through controlled methods. This keeps the system secure, modular, reusable and easy to maintain.

4 Objectives

1. Model a working ATM using OOP concepts
2. Provide secure, PIN-based authentication
3. Perform withdraw, deposit, balance enquiry and PIN change
4. Protect sensitive data using encapsulation
5. Handle errors and invalid inputs gracefully
6. Build a modular program that is easy to extend

5 Features

- Card and PIN login
- Balance enquiry
- Cash withdrawal
- Cash deposit
- PIN change
- Mini statement
- Input validation
- Card lock after 3 wrong PINs
- Safe exit

SECTION 6

Requirements

6 Functional requirements

FR1	Authenticate the user with card number and PIN
FR2	Block the card after 3 wrong PIN attempts
FR3	Show the current account balance
FR4	Withdraw cash only when the balance is sufficient
FR5	Deposit only amounts greater than zero
FR6	Change the PIN after verifying the old PIN
FR7	Show a mini statement of recent transactions
FR8	Let the user continue or exit safely

Non-functional requirements

Security
PIN and balance are private data, reachable only through methods.

Usability
A clear menu and plain messages guide every step.

Reliability
Invalid input never crashes the program.

Maintainability
Small, separate classes make changes easy.

Portability
Runs on Windows, Linux and macOS.

SECTIONS 7 AND 8

OOP Concepts and Class Design

7 OOP concepts used

Class and Object
Account, ATM and Transaction are classes; every customer account is an object.

Encapsulation
Balance and PIN are private and reached only through public methods.

Abstraction
The user sees simple options while the internal logic stays hidden.

Inheritance
Savings and Current accounts inherit from the base Account class.

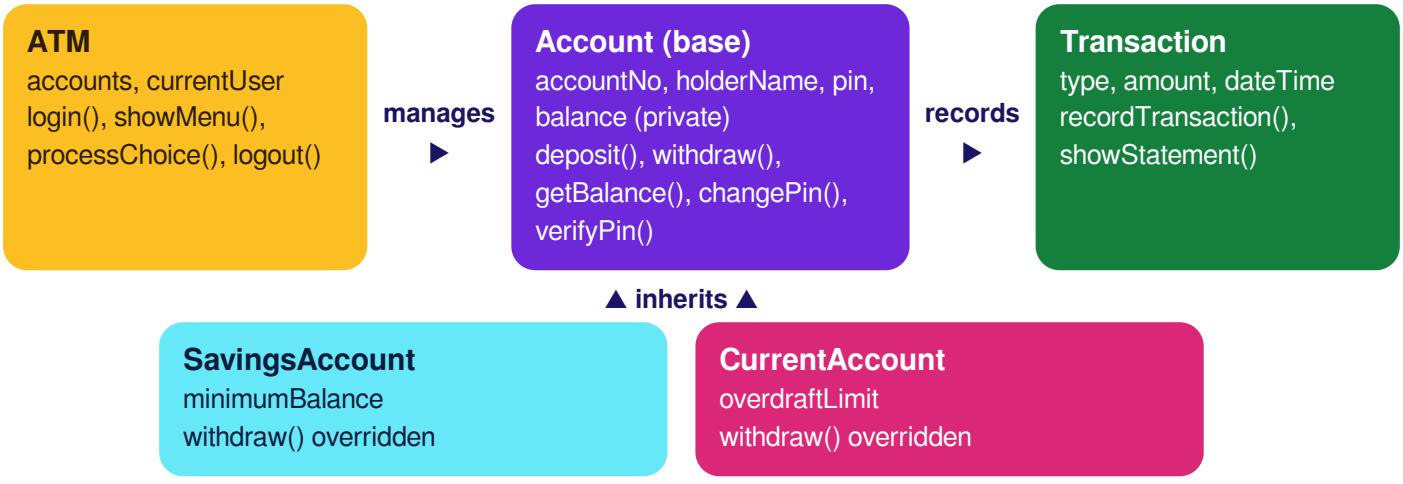
Polymorphism
withdraw() behaves differently for each account type.

Constructor
Sets up the account number, name, PIN and opening balance.

Exception handling
Covers insufficient funds, invalid PIN and wrong input.

7 ideas, one real-life system

8 Class design



Module Description

9 Module description

ATM

Controls the whole session: login, menu, routing of choices and logout.

Data: accounts, currentUser | **Methods:** login(), showMenu(), processChoice(), logout()

Account (base)

Holds customer data and the common banking operations.

Data: accountNo, holderName, pin, balance (all private) | **Methods:** deposit(), withdraw(), getBalance(), changePin(), verifyPin()

SavingsAccount

Account type with a minimum balance rule.

Data: minimumBalance | **Methods:** withdraw() overridden

CurrentAccount

Account type that allows an overdraft limit.

Data: overdraftLimit | **Methods:** withdraw() overridden

Transaction

Records every action for the mini statement.

Data: type, amount, dateTime | **Methods:** recordTransaction(), showStatement()

How the classes relate

Inheritance
Savings and Current accounts extend Account.

Association
The ATM manages many accounts.

Recording
Each account keeps many transactions.

SECTIONS 10 AND 11

Workflow and Technology

10 Workflow, step by step

1 START
The program starts and shows a welcome screen.

2 Enter card number
The user enters the account or card number.

3 Enter PIN
The user enters the 4-digit PIN.

4 Verify credentials
The system checks card number and PIN.

5 Valid?
No: retry, and after 3 wrong PINs the card is blocked. Yes: open the menu.

6 Main menu
Balance, withdraw, deposit, change PIN, statement, exit.

7 Perform operation
The chosen option is executed.

8 Update records
Balance and transaction history are saved.

9 Another transaction?
Yes: back to the menu. No: continue.

10 Exit
The card is released and the session ends.

Operation flows

Withdraw
Enter amount, check it is valid, check balance, deduct, record and show the new balance.

Deposit
Enter amount, check it is above zero, add it, record it and show the balance.

Change PIN
Verify the old PIN, enter the new PIN twice, update it if both match.

Mini statement
Read the stored transactions and show the latest with type, amount and date.

11 Technology and requirements

Language	C++ / Java	**OS**	Windows, Linux or macOS
IDE	Code::Blocks, Eclipse, IntelliJ, VS Code	**RAM**	2 GB or more

Operation Algorithms

12 Operation algorithms

Login

1. Read the card number
2. Find the matching account
3. Read the PIN
4. Call verifyPin()
5. Wrong: add 1 to attempts; at 3 block the card
6. Right: open the main menu

Withdraw

1. Read the amount
2. Reject zero or negative amounts
3. Check balance is enough for the account type
4. Deduct the amount
5. Record the transaction
6. Show the new balance

Deposit

1. Read the amount
2. Reject zero or negative amounts
3. Add the amount to the balance
4. Record the transaction
5. Show the updated balance

Change PIN

1. Read the old PIN and verify it
2. Read the new PIN twice
3. If both match, update the PIN
4. Confirm to the user
5. Otherwise show an error

Mini statement

1. Read the stored transactions
2. Take the latest entries
3. Show type, amount and date for each

Exit

1. Ask: another transaction?
2. Yes: return to the main menu
3. No: release the card and end the session

Testing and Sample Output

13 Test cases and expected results

T1	Correct card and PIN	**Login succeeds, menu opens**
T2	Wrong PIN once	**Error shown, user can retry**
T3	Wrong PIN three times	**Card is blocked**
T4	Withdraw within balance	**Balance reduced, transaction recorded**
T5	Withdraw above balance	**"Insufficient Balance" message**
T6	Deposit zero or negative	**Amount rejected**
T7	Change PIN with wrong old PIN	**Change refused**
T8	Invalid menu choice	**Menu is shown again**

14 Sample output

```
ATM SIMULATOR
Enter Card Number : 1001
Enter PIN : ****
Login successful

1 Balance 2 Withdraw 3 Deposit
4 Change PIN 5 Statement 6 Exit
Choice : 2
Enter amount : 500
Please collect your cash.
Another transaction? (y/n)
```

Illustrative output. Card number and amount are examples.

SECTIONS 15 TO 17

Advantages, Scope and Conclusion

15 Advantages

- ✓ Safe practice of banking logic with no real money
- ✓ Modular and easy-to-understand code
- ✓ Strong data security through encapsulation
- ✓ Easy to extend with new features and account types
- ✓ Clear demonstration of OOP concepts

16 Future scope

1. Graphical user interface for a realistic ATM screen
2. Database connectivity for permanent storage
3. Fund transfer between accounts
4. OTP-based verification
5. Multi-language support and receipt printing

17 Conclusion

The ATM Simulator models the essential operations of a real ATM using Object-Oriented Programming. It shows how classes, objects, encapsulation, inheritance and polymorphism combine into a secure and well-structured application. The project strengthens the understanding of OOP and gives a solid base for larger banking and transaction systems.

PREPARED BY

Ranjan Kumar Nandi

ID B125095 | CSE B(1) | 2nd year, 3rd semester

International Institute of Information Technology